

ShellCore初赛文档

小队成员：唐博文，冯孟熙，高铭均

目录

- 简介
- 工作量
 - 一些功能的简单介绍
- 调试/开发过程
- AI使用
- 总结与展望

简介

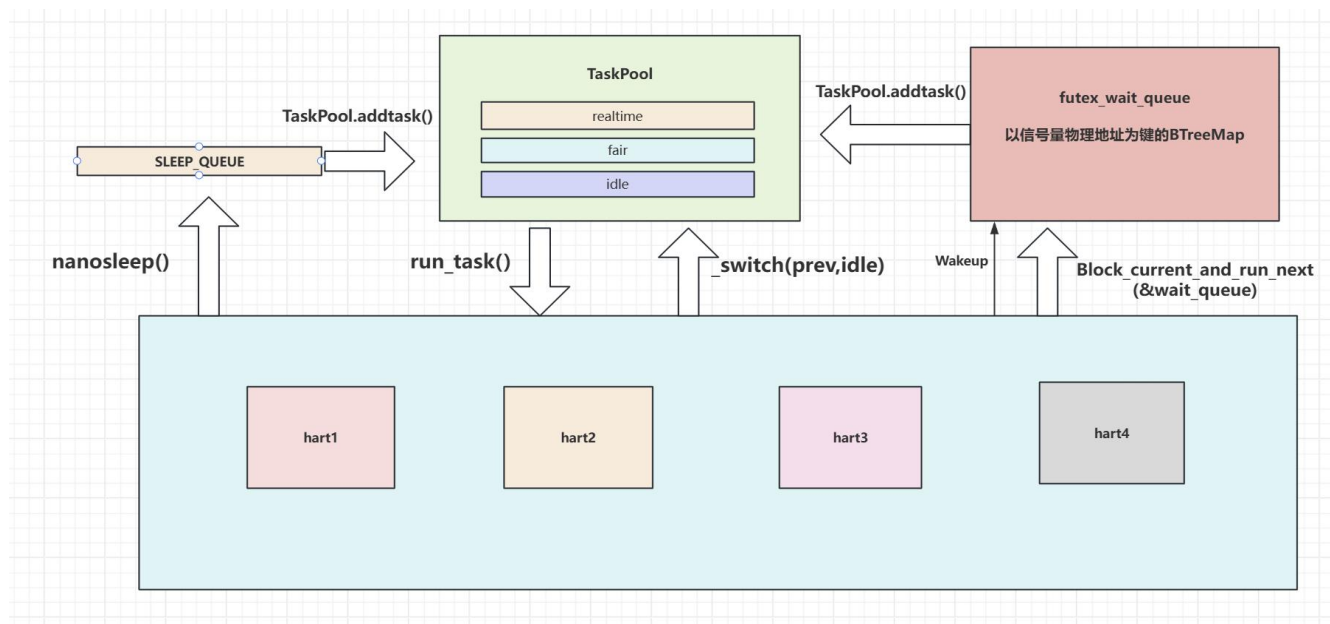
- ShellCore是三位成员以清华大学开源操作系统训练营中的教学内核**rCore ch7**开发而来
- 分工：
 - 唐博文：内存管理COW，文件系统ext4初始架构，多核进程/线程管理
 - 冯孟熙：龙芯架构兼容，SMP初始框架，内存管理与文件系统完善
 - 高铭均：网络栈实现，早期系统完善

工作量

- 自rCore ch7以来，截至目前，main分支已有**700+**个commit记录，详细记录了开发流程
- 代码量相较于rCore ch7也在不断增大
- 有很多rCore ch7没有的功能，例如：
 - 架构相关：支持loongarch
 - 进程相关：多线程模型，**多核调度，多级队列调度**，线程的阻塞状态
 - 内存相关：堆区支持，写时复制，基于**ASID**的页表刷新，mmap共享文件映射，栈区懒分配，大页等
 - 文件相关：支持ext4的extent扩展，**extent中间节点的缓存**，tmpfs，proc等虚拟文件的创建与访问，多级页目录等
 - io相关：MMIO，PCIE等
 - Linux语义：系统调用**错误码**对应，**信号处理**
 - 网络相关：**各类socket实现**，网卡驱动
- 具体可见手册以及展示视频
- 由于**手册**已经**详细**介绍了以上技术实现，初赛ppt只选择重点并进行简单介绍

多核调度与多级反馈队列

- 如图是我们的调度结构
- 四个核拥有自己的**本地队列**，其在进程切换时会调用 `run_tasks()`。
- 当本地队列**没任务或任务数量小于** `LOCAL_QUEUE_REFILL_TARGET`时，会向TaskPool索取任务。
- **本地队列**是FIFO，但TaskPool是根据**进程优先级**以及**调度策略**进行任务的分配的。
- TaskPool有三种队列，每种队列内都是一个**大端堆**。稳定 $\log_2 n$
- `futex_wait_queue`是用于信号量的等待队列，即系统调用 `futex`。是以**信号量物理地址**为键的BTree，物理地址保障了信号量队列的**唯一性**，当futex唤醒时，将任务插入到TaskPool中
- `SLEEP_QUEUE`是睡眠队列，每次**陷入内核态**会更新 `SLEEP_QUEUE`中睡眠队列的时间戳，当时间戳到达时，将任务放到TaskPool里



ASID

- 每次进程切换或陷入内核态后都要进行**TLB刷新**，否则TLB中缓存的页表项与原权限不符会导致**访址异常**
- 而TLB刷新会连带着TLB内所有的页表项都刷新，包括其它页表项，例如内核态的页表项其实本**不需要**刷新，这对RV和LA都会产生额外的开销
 - RV：访址时需要MMU**重新遍历**SV39三级页表
 - LA：需要操作系统指导loongarch进行**TLB重填**，开销相比RV更大
- 因此我们给页表分配了ASID，RV和LA都可以支持**仅同ASID页表**的TLB刷新
 - LA：sfence.vma
 - RV：invtlb
- 详见ShellCore初赛文档

错误码处理

- Linux系统调用会根据不同的执行过程返回不同的数值，这里列举一些我们对于**错误码**的设置，详见ShellCore初赛文档
 - EPERM = 1, /*不允许的操作*/
 - ENOENT = 2, /*没有那个文件或目录*/
 - ESRCH = 3, /*没有那个进程*/
 - EINTR = 4, /*中断的系统调用*/
 - EIO = 5, /*输入/输出错误*/

信号处理

- 以下的情况会进行**信号检测**
 - 进程从SLEEP恢复
 - 进程从BLOCK恢复
 - 进程陷入内核态
- 信号处理符合Linux语义（详见初赛文档）
 - `const SIGHUP = 1 << (1 - 1); // 终端挂断`
 - `const SIGINT = 1 << (2 - 1); // 键盘中断 (Ctrl+C)`
 - `const SIGQUIT = 1 << (3 - 1); // 键盘退出 (Ctrl+\)`
 - `const SIGILL = 1 << (4 - 1); // 非法指令`
 - `const SIGTRAP = 1 << (5 - 1); // 断点/陷阱指令`
 - `const SIGABRT = 1 << (6 - 1); // 异常中止 (Abort)`
- 我们同样支持了用户程序**自定义**信号执行流

各类Socket

- 完成了udp/tcp/的实现，raw/unix/StandardNetlink socket的简单基本支持
- StandardNetlink完成了
 - 符合linux内核标准的LinkReplyPacket拼接和返回
 - 给系统添加ip地址
- 借由引用的smoltcp依赖完成：
 - 对于各类报文的解析
 - 对于底层网卡设备的驱动
 - 为上层的调用提供可用的socket
- 为各类socket实现了File特性，可以像处理文件一样对socket进行读写处理
- 详见ShellCore初赛文档

调试过程

- 出现用户态缺页等内存问题时：
 - **连续打印系统调用号与返回值**
 - 判断是重分配？未分配越界？耗尽等问题
 - 在mmap/munmap/brk，后打印指定进程memory_set中所有的**逻辑段**，判断进程内存**前后变化的过程**结合系统调用号判断行为进行调试
 - 快速定位出错系统调用后，使用**gdb**调试
- 出现死锁问题时：
 - **连续打印系统调用号与返回值**
 - 找到卡死的系统调用，使用gdb到该系统调用处进行处理，检测是哪个变量的锁**未释放**，进行调试
- 过程中也遇到了些**文件找不到/格式不对**，或者发送的信号被**忽略**等问题，但不算典型以及常见问题。调试过程也类似上述，结合用户程序或musl/glibc库源码，以及系统调用**返回值**判断哪个系统调用不符合预期，进行调试

AI使用

- 工具：Deepseek v4 pro（posix标准文献查找与翻译），GPT5.5(部分代码生成)
- **使用过程**举例：
 - 对于cyclictest_testcode.sh中的程序，指定程序是由哪个文件里的源码编译而来，其运行指令的外部参数对应的执行流是源码的哪些？对应musl/glibc库函数在musl/glibc的哪个文件里？使用了哪些系统调用”
 - “140号系统调用的效果是什么？其不同选项对应的效果是什么？”
 - “我在tcb里新加了两个字段sched_police以及sched_priority，对于140号系统调用set_priority，先进行参数检查，然后设置优先级”
 - 对于用户注册的信号处理，似乎用户态要进行上下文的修改，这样的操作是否安全？既然如此，我是否应该去用户态地址空间读取新的上下文？
- 我们声明：LLM在我们的项目中只用于**重复/低效**工作的替代，并且所有用AI生成的代码均经**严格**的人工审核。

总结与展望

- 总体上，目前我们对于Linux的理解还有待加深，我们的内核也还不够健壮（尤其是多核场景下）。并且由于队员是第一次进行os内核的开发，甚至部分队员是本学期刚刚学习此课程，我们都没有在开发初期在**更高视角**去观察内核的设计，因此实际上有时也不理解Linux的一些代码**为什么**如此设计的原因，不过随着开发进度不断加深，也更加了解了内核全貌，积累了**经验**吧。
- 但似乎测试用例刷的也**不够多**，准备在下一阶段进行一个**补充**
- 下阶段的任务有：
 - 进行代码的**去耦合化**以及接口的**重构**，增加可读性，。
 - 完成一些自己的想法，例如：较好的**调度算法**，**图形栈**支持，更多符合linux标准的设计等。
 - 进行针对开发板的驱动开发。

谢谢